DEEP LEARNING

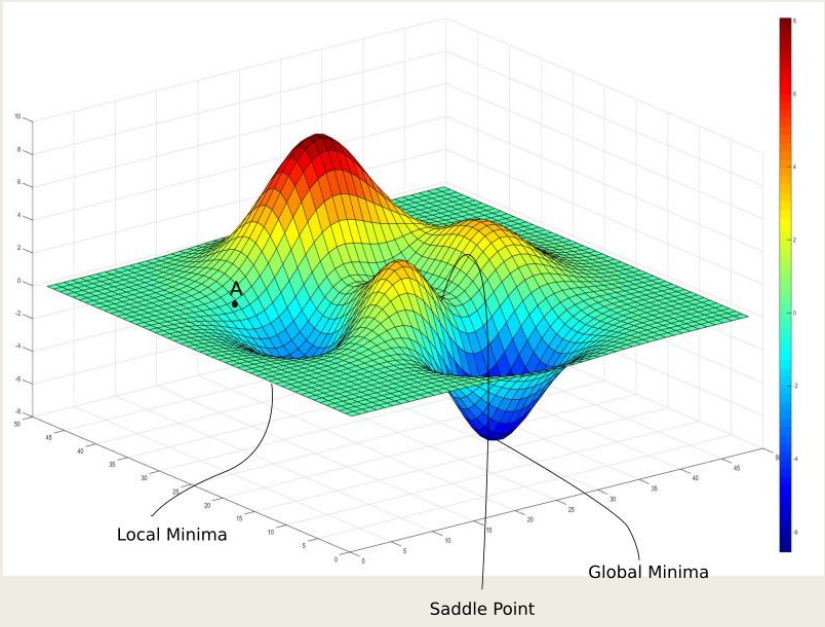
Lecture 6

Backpropagation and Gradient Descent

Instructor: Zafar Iqbal

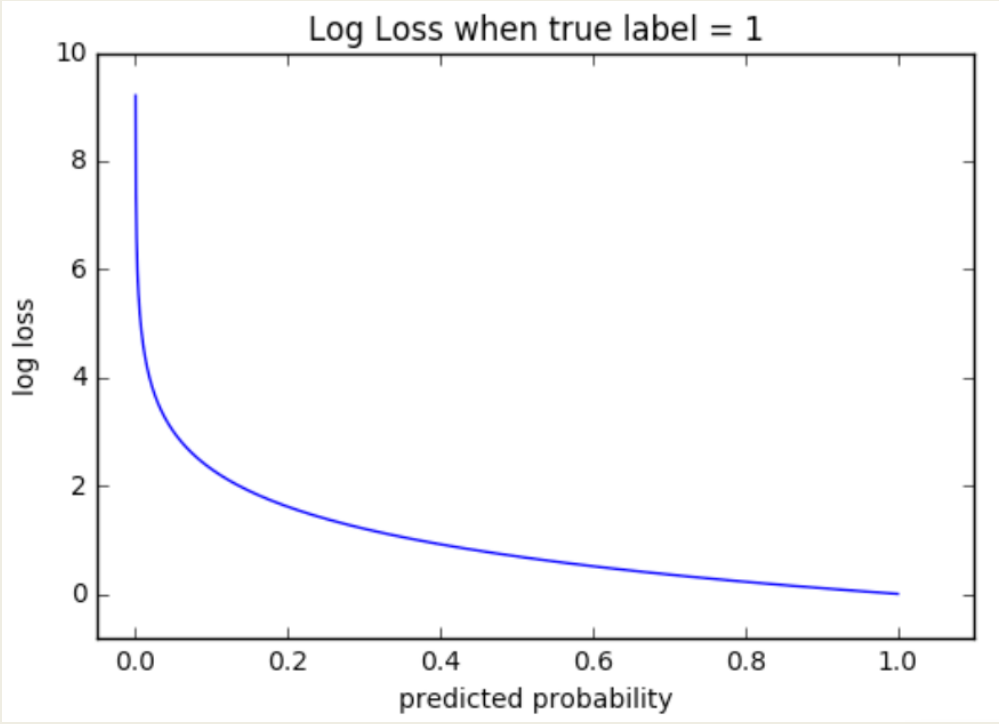
Agenda

- Introduction
- Loss function
- Gradient Descent
- Types of Gradient Descent
- Chain Rule
- Vanishing and exploding gradient



Introduction

- Neural networks learn by minimizing error between predictions and targets.
- Backpropagation and Gradient Descent are the core learning algorithms.
- Enable models to automatically adjust weights for better accuracy.



The Role of Loss Function

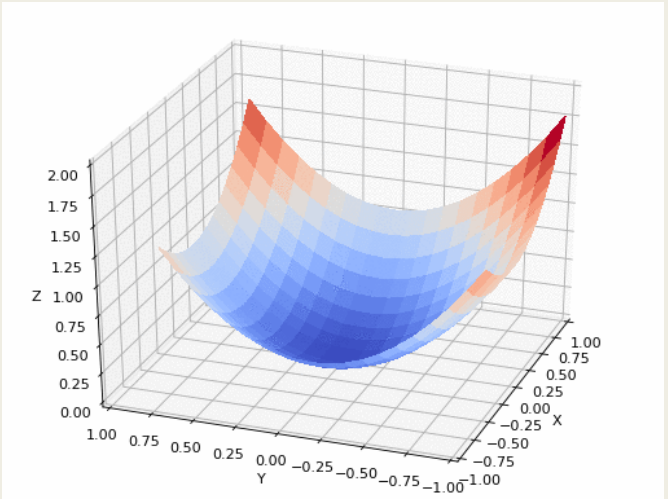
- When a model makes predictions, we need a way to measure how wrong it is - just like marking exam answers.
- This “wrongness score” is called the Loss.
- Purpose of Loss
  - *It quantifies the gap between:*
    Predictions vs. True Labels
- The training goal is:
  - *Adjust weights → reduce loss → improve predictions*
- So:
  - High Loss → Poor Model
  - Low Loss → Better Model

Different Losses for Different Problems?

- Regression (Predicting Numbers)
- Examples:
  - *Predict house price*
  - *Predict temperature*
- Outputs are continuous numbers, so we use Mean Squared Error (MSE)
- $MSE = average\ of\ (prediction - actual)^2$ $L = \frac{1}{n}\sum(y - \hat{y})^2$
- Punishes large mistakes more (because squared)
  Smooth and differentiable → easy for gradient descent
  Works well when errors are like small noise
- Classification (Predicting Categories)
- Examples:
  - *Cat vs Dog*
  - *Spam vs Not Spam*
- Outputs are probabilities (e.g., 0.9 cat, 0.1 dog)
- So we use Cross-Entropy Loss (Log Loss) $L = -\sum y \log(\hat{y})$
- It measures:
- How different the predicted probability distribution is from the true labels

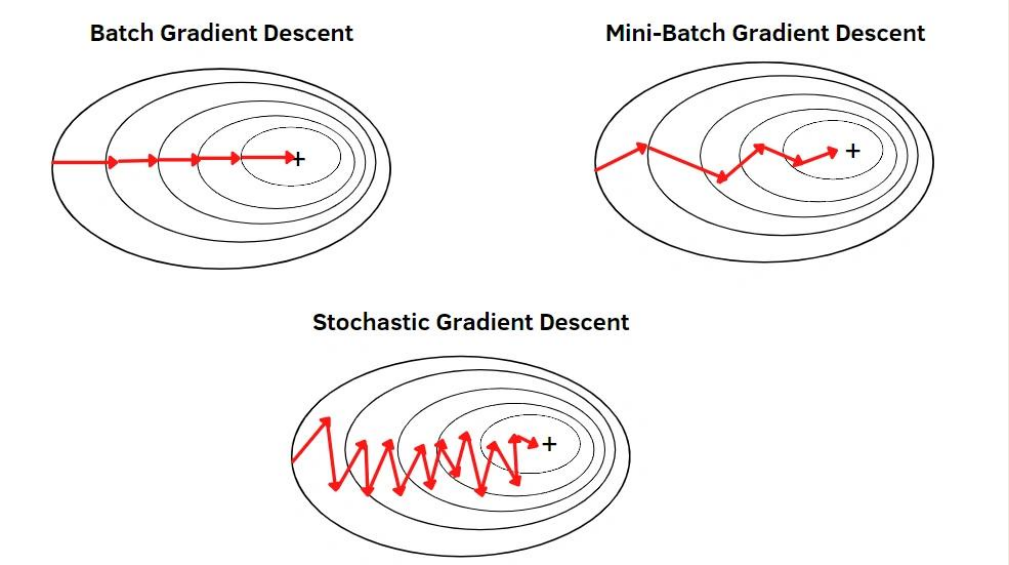
Intuition Behind Gradient Descent

- Optimization algorithm to minimize loss.
- Uses the gradient (slope) to update weights in the direction of steepest descent.
- Optimization is like walking down a hill
- Update rule:
  - $\mathrm{w_{new}} = \mathrm{w_{old}} - \eta \frac{\partial \mathrm{L}}{\partial \mathrm{w}}$
- Where
  - η : learning rate
  - $\frac{\partial \mathrm{L}}{\partial \mathrm{w}}$: gradient w.r.t. weight
- We *subtract* the gradient to minimize loss.



Types of Gradient Descent

- **Batch GD**
  - *Uses ALL training data to update once — stable but slow*
- **Stochastic GD (SGD)**
  - *Updates after each sample — fast, but noisy*
- **Mini-Batch GD**
  - *Best of both — updates on batches (e.g., size 32–256)*



Understanding Types of Gradient Descent

- We will use a simple regression problem so the calculations are easy to follow:
  - $\hat{y} = wx + b$
- Loss = Mean Squared Error
  - $L = -\frac{1}{N}\sum_{i=1}^{N} x_i\,(y_i - \hat{y}_i)^2$
- Gradients:
  - $\frac{\partial L}{\partial w} = -\frac{2}{N}\sum x\,(y - \hat{y})$
  - $\frac{\partial L}{\partial b} = -\frac{2}{N}\sum x\,(y - \hat{y})$
- We'll use the same tiny dataset throughout:

x	y
1	2
2	4
3	6

- Clearly the true relationship is $y = 2x$
- Start with:
- $w = 0, b = 0, \eta = 0.1$

Batch Gradient Descent (BGD)

Step-by-Step Calculation

- Predictions:
  - $\hat{y} = 0$
- Errors:
  - $e = y - \hat{y} = [2, 4, 6]$
- Compute gradients:
- $\frac{\partial L}{\partial w} = -\frac{2}{3}(1 * 2 + 2 * 4 + 3 * 6) = -\frac{2}{3}(2 + 8 + 18) = -\frac{2}{3}(28) = -18.67$
- $\frac{\partial L}{\partial b} = -\frac{2}{3}(2 + 4 + 6) = -\frac{2}{3}(12) = -8$

Update

- $w \leftarrow w - \eta \frac{\partial L}{\partial w} = 0 - 0.1(-18.67) = 1.867$
- $b \leftarrow b - \eta \frac{\partial L}{\partial b} = 0 - 0.1(-8) = 0.8$
- One update after full dataset

Batch Gradient Descent - Code

```
import numpy as np

X = np.array([1,2,3], dtype=float)
y = np.array([2,4,6], dtype=float)

w, b = 0.0, 0.0
lr = 0.1

for epoch in range(100):
    y_pred = w*X + b
    error = y - y_pred

    dw = -2/len(X) * np.sum(X * error)
    db = -2/len(X) * np.sum(error)

    w -= lr * dw
    b -= lr * db

print(w, b)
```

Stochastic Gradient Descent (SGD)

- Updates after every single sample

Step-by-Step (First 2 samples)

- Start:
  - $w = 0,\ b = 0$

Sample 1: (x=1, y=2)

- $\hat{y}$ =0*x+0=0
- $\hat{y} = 0$
- error = 2
- $\frac{\partial L}{\partial w} = -2x(y - \hat{y})$
  - $dw = -2(1)(2) = -4$
  - $db = -2(2) = -4$
- **Update:**
- Formula:
  - $new\ w = old\ w - \eta \cdot dw$
  - $w = 0 - 0.1(-4) = 0.4$
  - $b = 0 - 0.1(-4) = 0.4$

- **Sample 2: (x=2, y=4)**
  - $\hat{y} = 0.4 * 2 + 0.4 = 1.2$
  - *error = 4 – 1.2 = 2.8*
  - $dw = -2(2)(2.8) = -11.2$
  - $db = -2(2.8) = -5.6$
- Update:
  - $w = 0.4 - 0.1(-11.2) = 1.52$
  - $b = 0.4 - 0.1(-5.6) = 0.96$
- Model changes **after every data point**

Stochastic Gradient Descent-Code

```
import numpy as np

X = np.array([1,2,3], dtype=float)
y = np.array([2,4,6], dtype=float)

w, b = 0.0, 0.0
lr = 0.1

for epoch in range(100):
    for xi, yi in zip(X, y):
        y_pred = w*xi + b
        error = yi - y_pred

        dw = -2 * xi * error
        db = -2 * error

        w -= lr * dw
        b -= lr * db

print(w, b)
```

Mini-Batch Gradient Descent

- Let batch size = 2
  - Dataset:
  - Batch 1 → (1,2), (2,4)
  - Batch 2 → (3,6)
- **Batch 1 Calculation** (1,2), (2,4)
- Start:
  - $\mathrm{w} = 0,\ \mathrm{b} = 0$
- Errors:
  - $\mathrm{e} = [2, 4]$
  - $\mathrm{dw} = -\frac{2}{2}(1 * 2 + 2 * 4) = -1(2 + 8) = -10$
  - $\mathrm{db} = -\frac{2}{2}(2 + 4) = -6$
- Update:
  - $\mathrm{w} = 0 - 0.1(-10) = 1$
  - $\mathrm{b} = 0 - 0.1(-6) = 0.6$

Mini-Batch Gradient Descent

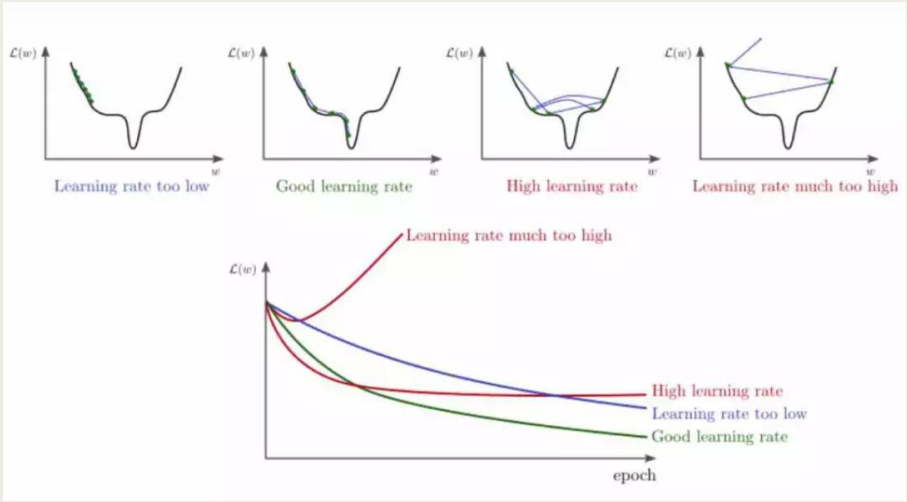
Batch 2 Calculation (3,6)

- **Predict**
  - $\hat{y} = 1 \cdot 3 + 0.6 = 3.6$
- **Error**
  - $e = 6 - 3.6 = 2.4$
- **Gradients**
- wrt w
  - $dw = -2x(y - \hat{y})$
  - $dw = -2(3)(2.4)$
  - $dw = -14.4$
- wrt b
  - $db = -2(2.4)$
  - $db = -4.8$

- **Update**
  - $w = 1 - 0.1(-14.4) = 1 + 1.44 = 2.44$
  - $b = 0.6 - 0.1(-4.8) = 0.6 + 0.48 = 1.08$

Final Model (After ALL Batches)

- $w = 2.44, b = 1.08$
- Your trained model:
- $\hat{y} = 2.44x + 1.08$



Mini-Batch Gradient Descent

```
import numpy as np
# data
X = np.array([1,2,3])
Y = np.array([2,4,6])
# parameters
w = 0
b = 0
lr = 0.1
batch_size = 2
def predict(x):
    return w*x + b
for start in range(0, len(X), batch_size):
    end = start + batch_size
    x_batch = X[start:end]
    y_batch = Y[start:end]
    y_pred = w*x_batch + b
    error = y_batch - y_pred
    dw = -2/len(x_batch) * np.sum(x_batch * error)
    db = -2/len(x_batch) * np.sum(error)
    w = w - lr*dw
    b = b - lr*db
    print(f"Batch {start//batch_size + 1}: w={w:.2f}, b={b:.2f}")
```

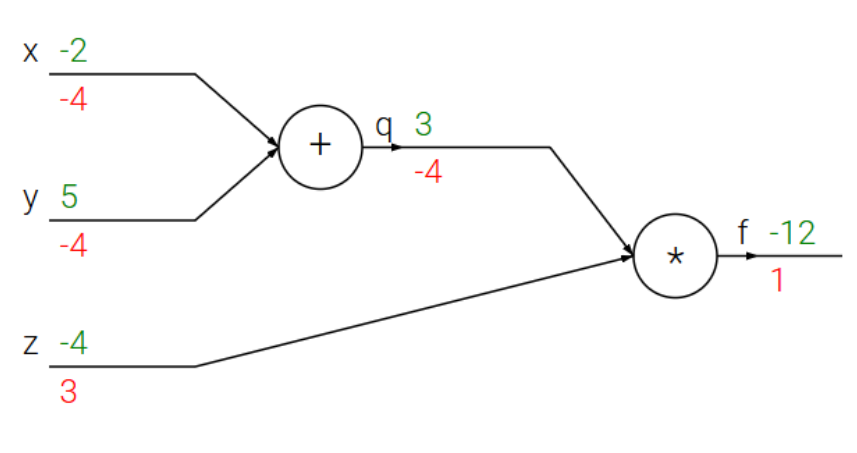
Gradient Descent Variants

- **Momentum**
  - *Smooths updates using moving average.*
- **RMSProp**
  - *Adapts learning rate by recent gradients.*
- **Adam**
  - *Combines Momentum + RMSProp (most common).*

The Need for Backpropagation

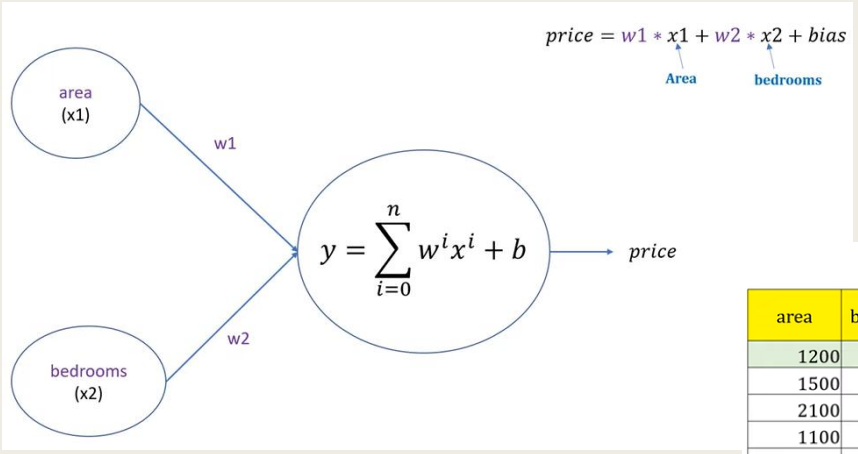
- We need gradients for **each weight** in the network.
- Backprop efficiently computes these gradients using the **chain rule of calculus**.

- **Forward Pass**: compute predictions and loss.
- **Backward Pass**: compute partial derivatives of loss w.r.t. each weight.
- **Weight Update**: apply gradient descent step.

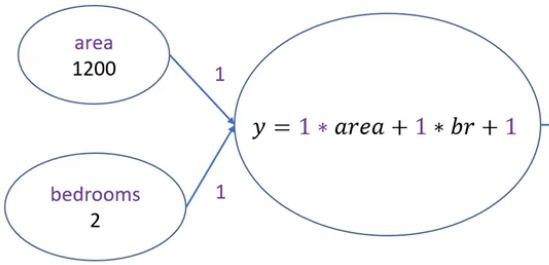


Chain Rule in Action

- Enables computing gradients layer by layer.
- Example (3-layer NN):
- $$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_3}\frac{\partial a_3}{\partial a_2}\frac{\partial a_2}{\partial a_1}\frac{\partial a_1}{\partial w_1}$$
- Understanding Chain rule with example



area	bedrooms	price
1200	2	70
1500	3	120
2100	4	230
1100	3	105
1300	2	90
900	1	55

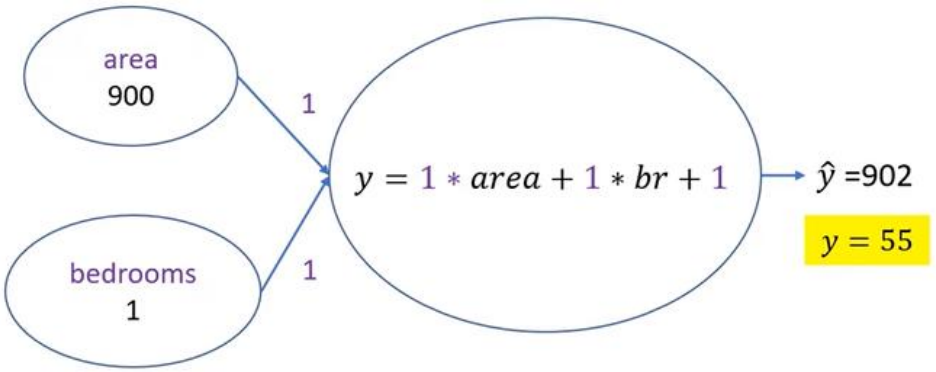


$error1 = y - \hat{y}$ = 1133

$squared\ error1$ = 1283689

Chain Rule

area	bedrooms	price
1200	2	70
1500	3	120
2100	4	230
1100	3	105
1300	2	90
900	1	55



$error6 = y - \hat{y}$ = 847

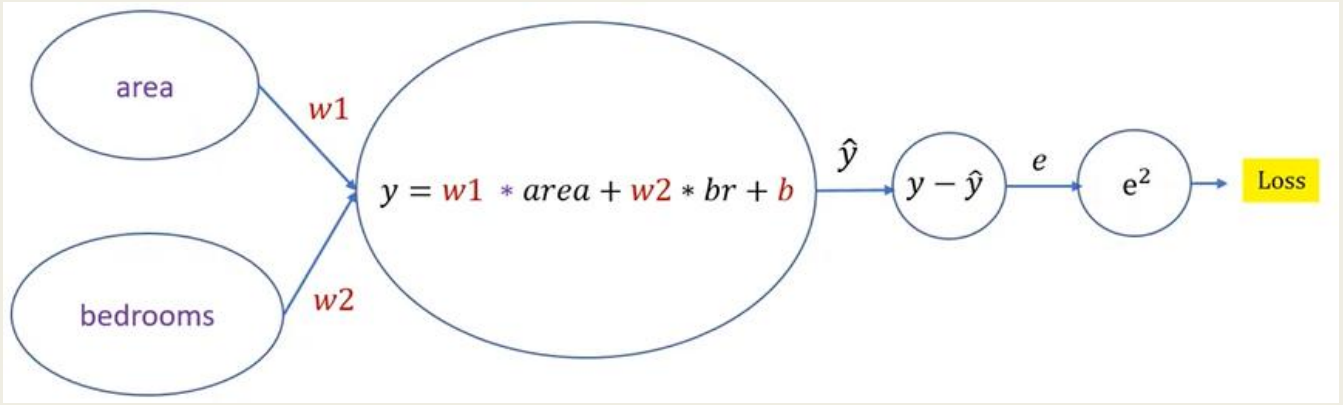
$squared\ error6$ = 717409

Total squared error = $sq\ error_1 + \cdots + sq\ error_6$

Mean squared error = $(sq\ error_1 + \cdots + sq\ error_6) / 6$

$$Loss = Mean\ Squared\ Error$$

Chain Rule



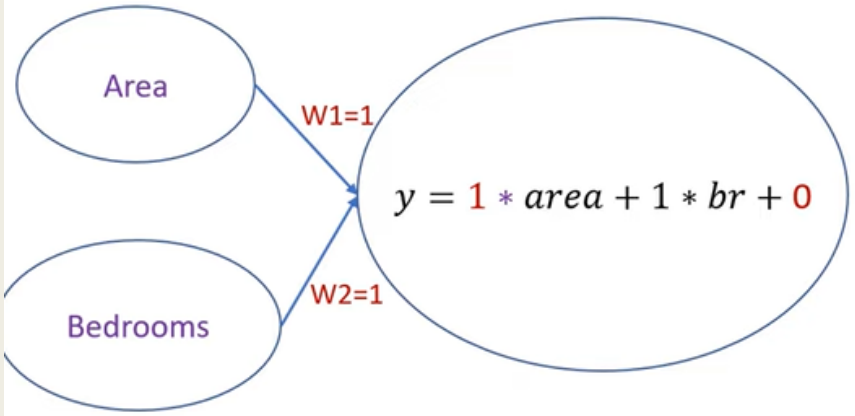
$$w1 = w1 - something$$

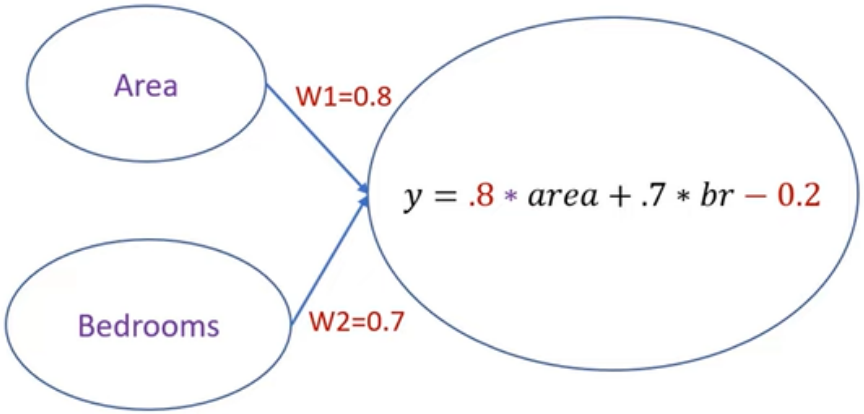
$$w1 = w1 - learning\ rate * {}^{\partial}/_{\partial w1}$$

$$w2 = w2 - learning\ rate * {}^{\partial}/_{\partial w2}$$

$$b = b - learning\ rate * {}^{\partial}/_{\partial b}$$

Chain Rule





$$w1 = w1 - learning\ rate * \partial/\partial w1$$

W1 = 1- 0.2 = 0.8

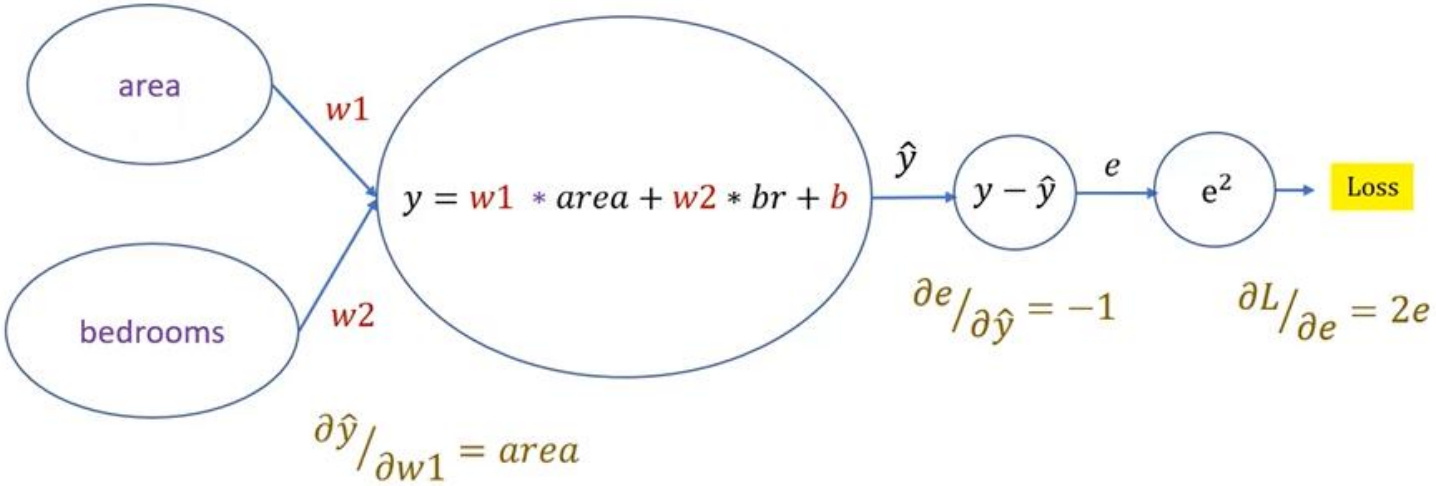
$$w2 = w2 - learning\ rate * \partial/\partial w2$$

W2 = 1- 0.3 = 0.7

$$b = b - learning\ rate * \partial/\partial b$$

b = 0 – 0.2 = -0.2

Chain Rule



$\partial L / \partial w1 = ?$

$\partial L / \partial w1 = \partial L / \partial e * \partial e / \partial \hat{y} * \partial \hat{y} / \partial w1$

Chain Rule

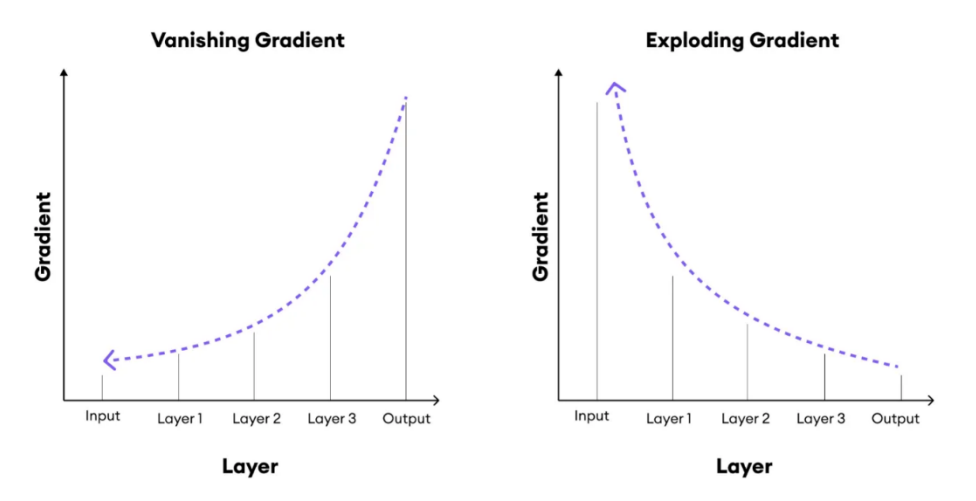
$\partial L / \partial w1 = -2e * area$

Vanishing and Exploding Gradients

Vanishing Gradient (Very Small Updates)

- The learning signal becomes **too tiny**, so the network **almost stops learning.**
- In deep networks, when gradients are repeatedly multiplied by small numbers (like 0.1), they keep shrinking…
- 0.1 → 0.01 → 0.001 → 0.0001 → **almost zero**
- Earlier layers learn VERY slowly
- Weights barely change
- Training gets stuck

"Imagine passing a message through 20 friends, each whispering quietly.
By the time it reaches the last friend — the message is almost gone."



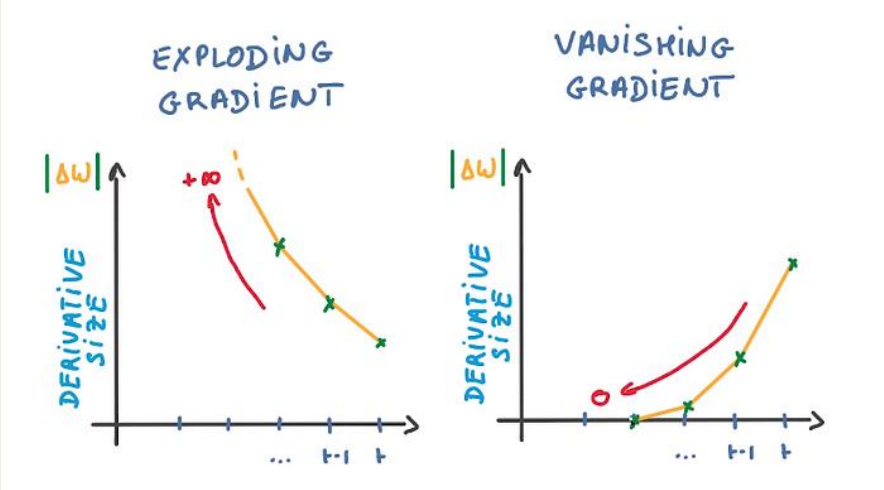
Vanishing and Exploding Gradients

Exploding Gradient (Very Large Updates)

- The learning signal becomes **too big**, so the weights **change wildly** and training blows up.
- If gradients keep multiplying by large numbers (like 5):
  5 → 25 → 125 → 625 → **huge numbers**
  - *Weights become extremely large*
  - *Loss becomes NaN*
  - *Model behaves randomly*
  - *Training fails*

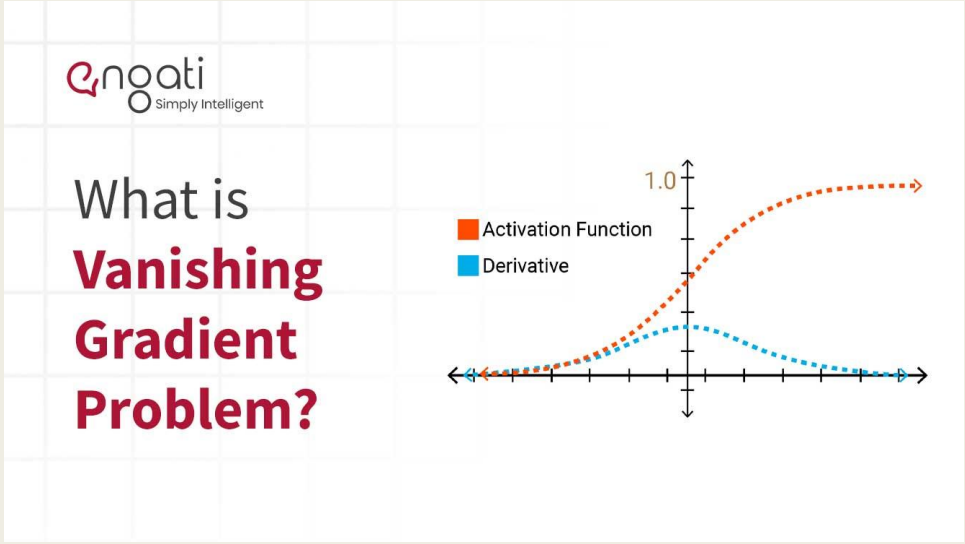
"Imagine turning the steering wheel too much while driving —

the car goes out of control."



Techniques to Handle Gradient Issues

- Use ReLU instead of sigmoid/tanh
- Use Batch Normalization
- Use Residual Networks (ResNets)
- Use Gradient Clipping (for exploding gradients)
- Use proper weight initialization



Task - Predict the Next Step

- A model is trying to learn the equation:
  - $y = wx$
- Right now the weight is:
  - $w = 2$
- The gradient of the loss with respect to the weight is:
  - $\frac{dL}{dw} = 6$
- The learning rate is:
  - $\alpha = 0.1$
- **Question:**
- Using gradient descent, calculate the **new value of w** after one update.
- Use the formula:
  - $w_{new} = w - \alpha \times \frac{dL}{dw}$

Task - Solution

- We are given:
  - $w = 2, \frac{dL}{dw} = 6, \alpha = 0.1$
- Gradient Descent update rule:
  - $w_{new} = w - \alpha \times \frac{dL}{dw}$
- Substitute values:
  - $w_{new} = 2 - 0.1 \times 6$
- Calculate:
  - $0.1 \times 6 = 0.6$
- So:
  - $w_{new} = 2 - 0.6 = 1.4$
- The new value of $w$will be: 1.4
- *Since the gradient is positive, we move left (subtract). The learning rate controls how big the step is. So the weight decreases from 2 to 1.4.*